

PROJECT DOCUMENTATION

AI FITTRACK

TECHNOLOGY STACK

Node.js • Express.js • MongoDB Atlas • Mongoose • JWT • bcrypt.js • Google Gemini AI • Postman

1.INTRODUCTION

Project Title:

AI FitTrack – Personalized Fitness Recommendations Powered by AI

Project Description :

AI FitTrack is a secure, scalable, AI-powered RESTful backend application developed to simplify workout tracking and provide intelligent fitness guidance. The backend is built using Node.js and Express.js, uses MongoDB with Mongoose for persistence, and integrates Google Gemini AI for personalized workout recommendations and fitness insights.

The application follows the Model–View–Controller (MVC) architecture. JWT-based authentication protects private resources, while bcrypt.js is used to securely hash passwords before storage. The API returns structured JSON responses so it can be consumed by a React application, mobile application, or API testing tool such as Postman.

Team information:

Team leader:

PRIYADHARSHINI M(pd260202@gmail.com)

Team members:

1.ELLEN CHRISTINA G (ellenchristyemily@gmail.com)

2.SATHIYAPRIYA R(rsathiyapriya1717@gmail.com)

3.SIVASAKTHI V (sivasakthi79142@gmail.com)

2. Project Overview

AI FitTrack is an AI-powered fitness tracking and recommendation system designed to help users manage their workout activities and receive personalized fitness guidance.

The application provides a RESTful backend using **Node.js**, **Express.js**, **MongoDB**, and **Mongoose**. It uses **JWT authentication** for secure access and

bcrypt.js for password protection. Google Gemini is integrated to generate personalized workout recommendations and fitness insights.

Main Features

- User registration and login
- JWT-based authentication
- User profile management
- Workout creation, viewing, updating, and deletion
- Workout search
- AI-powered workout recommendations
- AI-powered fitness insights
- Centralized error handling
- REST API architecture

3. Architecture

AI FitTrack follows a layered architecture based on the MVC design pattern. The system separates routing, business logic, authentication, database operations, and AI services into independent modules.

When a client sends an HTTP request, Express receives it and forwards it to the appropriate route. Protected routes pass through authentication middleware, which validates the JWT. Controllers process validated requests and invoke database models or AI services. Mongoose handles MongoDB operations, while a dedicated service layer communicates with Google Gemini. The controller finally returns a structured JSON response.

Architecture Components

1. Client Layer

The client layer is the entry point. It can be a React application, mobile application, or Postman. It sends API requests, stores the JWT token after login, invokes workout and AI APIs, and displays JSON responses.

2. Express Server Gateway

The Express server starts the HTTP server, parses JSON requests, enables CORS, registers routes, connects middleware, and participates in centralized error handling.

3. Authentication Middleware

The middleware reads the Authorization header, extracts the JWT, verifies its signature, decodes user information, attaches it to the request, and rejects invalid or missing tokens.

4. Route Layer

Routes define endpoints for authentication, workouts, and AI features. They connect incoming HTTP requests to controller methods.

5. Controller Layer

Controllers validate input, execute application logic, invoke models or services, and generate standardized JSON responses.

6. Service Layer

The service layer contains reusable logic. The Gemini service prepares prompts and receives AI responses; the JWT service creates and verifies tokens; the password service hashes and compares passwords.

7. Database Layer

MongoDB stores persistent application data and Mongoose provides schema definitions, validation, and database operations.

4. Setup Instructions

Prerequisites

Install the following software:

- Node.js
- MongoDB
- npm
- Postman or Thunder Client
- Google Gemini API key

Installation

Clone or download the project and open the project folder in a terminal.

Install the required dependencies:

```
npm install
```

Environment Configuration

Create a .env file and configure the following variables:

PORT=5000

MONGO_URI=mongodb://localhost:27017/aifittrack

JWT_SECRET=your_secret_key

GEMINI_API_KEY=your_google_gemini_api_key

Replace the placeholder values with the appropriate configuration for your environment.

5. Folder Structure

The project follows a modular backend structure

Folder Structure	
Folder / File	Purpose
 controllers/	Contains request handlers and business logic for each feature.
 authController.js	Handles authentication logic.
 workoutController.js	Handles workout-related operations.
 aiController.js	Handles AI recommendation and insights.
 models/	Defines Mongoose schemas for MongoDB.
 User.js	User schema and model.
 Workout.js	Workout schema and model.
 routes/	Defines API endpoints for different features.
 authRoutes.js	Authentication routes.
 workoutRoutes.js	Workout management routes.
 aiRoutes.js	AI recommendation and insights routes.
 middleware/	Contains middleware functions.
 authMiddleware.js	Verifies JWT tokens for protected routes.
 errorMiddleware.js	Handles errors and sends proper responses.
 services/	Reusable service functions, including Gemini AI integration.
 geminiService.js	Handles Google Gemini API calls.
 config/	Database and configuration setup.
 database.js	Connects to MongoDB.
 .env	Environment variables (not committed to Git).
 package.json	Project dependencies and scripts.
 server.js	Starts the HTTP server and database connection.

6. Running the Application

Step 1: Start MongoDB

Make sure MongoDB is running on the system.

Step 2: Install Dependencies

```
npm install
```

Step 3: Configure Environment Variables

Ensure that the .env file contains the required MongoDB, JWT, and Gemini API configuration.

Step 4: Start the Server

npm start

The server runs on the configured port, for example:

http://localhost:5000

The API can then be tested using **Postman** or **Thunder Client**.

7. API Documentation

 Authentication API Endpoints for user registration, login and profile management.		
Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Login and receive JWT
GET	/api/auth/profile	Get authenticated user profile

POST /api/auth/register

Register a new user

Request Body (JSON)

```
{  "name": "John Doe",  "email": "john@example.com",  "password": "securePassword123"}
```

Response (201 Created)

```
{  "message": "User registered successfully",  "user": {    "id": 1,    "name": "John Doe",    "email": "john@example.com"  } }
```

POST /api/auth/login

Login and receive JWT

Request Body (JSON)

```
{  "email": "john@example.com",  "password": "securePassword123"}
```

Response (200 OK)

```
{  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...  "user": {    "id": 1,    "name": "John Doe",    "email": "john@example.com"  } }
```

Include the token in the Authorization header for protected routes:
Authorization: Bearer <token>

GET /api/auth/profile

Get authenticated user profile

Headers

```
Authorization: Bearer <token>
```

Response (200 OK)

```
{  "id": 1,  "name": "John Doe",  "email": "john@example.com"}
```

8. Authentication

AI FitTrack uses **JWT (JSON Web Token)** for authentication.

Registration

The user provides:

- Name
- Email
- Password

The password is securely hashed using **bcrypt.js** before being stored.

Login

The user provides their email and password. After successful authentication, the server generates a JWT token.

The token is then sent with protected API requests.

Example:

Authorization: Bearer <JWT_TOKEN>

Protected Resources

Authenticated users can access protected resources such as:

- User profile
- Workout records
- AI recommendations
- AI fitness insights

9. User Interface

The AI FitTrack system is designed to provide a simple interface for fitness management.

The user interface can provide the following sections:

Login / Registration

Users can create an account and securely log in.

Dashboard

The dashboard can display:

- Total workouts
- Workout duration
- Calories burned
- Recent workout activity
- AI-generated insights

Workout Management

Users can:

- Add workouts
- View workouts
- Edit workouts

- Delete workouts
- Search workout records

AI Recommendation

Users provide information such as:

- Age
- Fitness goal
- Experience level

The AI generates a personalized weekly workout plan.

Fitness Insights

The system can analyze:

- Total workouts
- Average workout duration
- Total calories burned

and generate fitness progress information and suggestions.

10. Testing

The APIs can be tested using **Postman** or **Thunder Client**.

Authentication Testing

Test:

- Successful registration
- Duplicate email registration
- Successful login
- Invalid login credentials
- Access with valid JWT
- Access without JWT

Workout Testing

Test:

- Create workout
- Retrieve workouts

- Retrieve workout by ID
- Update workout
- Delete workout
- Invalid workout ID
- Unauthorized access

AI Testing

Test:

- AI workout recommendation
- AI fitness insights
- Valid input
- Missing input
- Invalid input
- Gemini API response handling

Error Testing

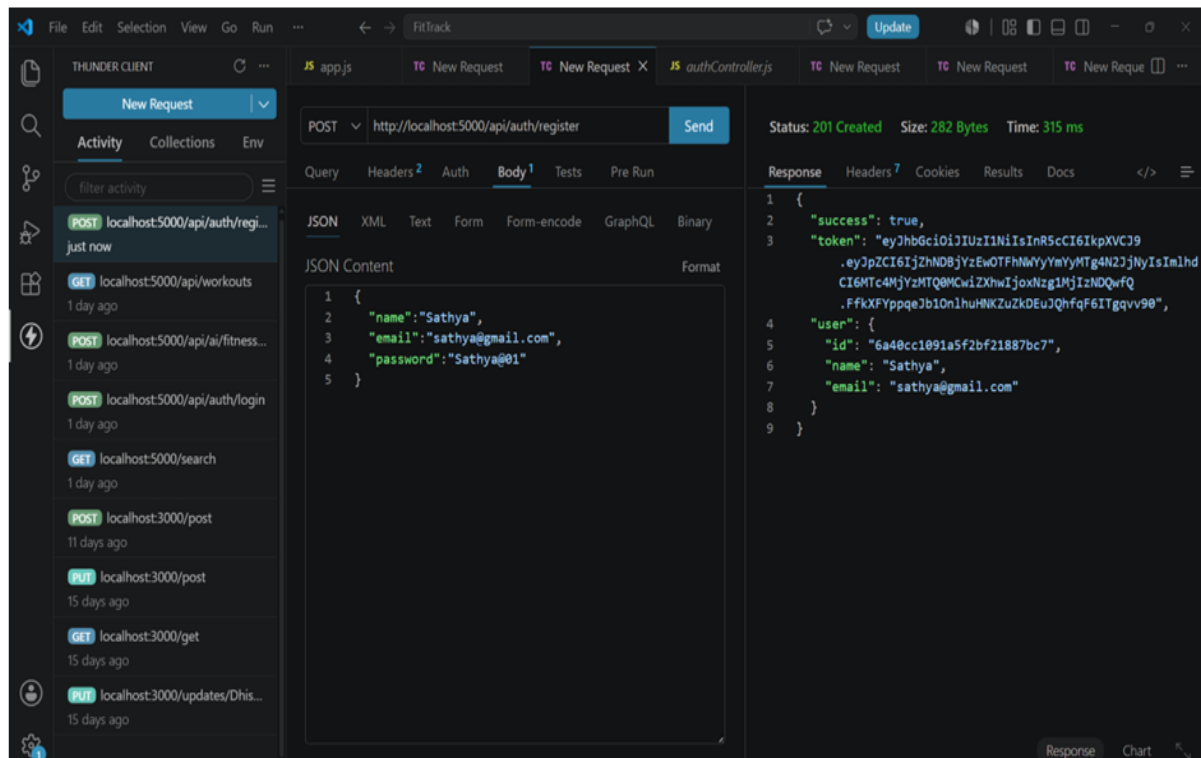
Test:

- Invalid routes
- Invalid request data
- Database errors
- Authentication errors
- Server errors

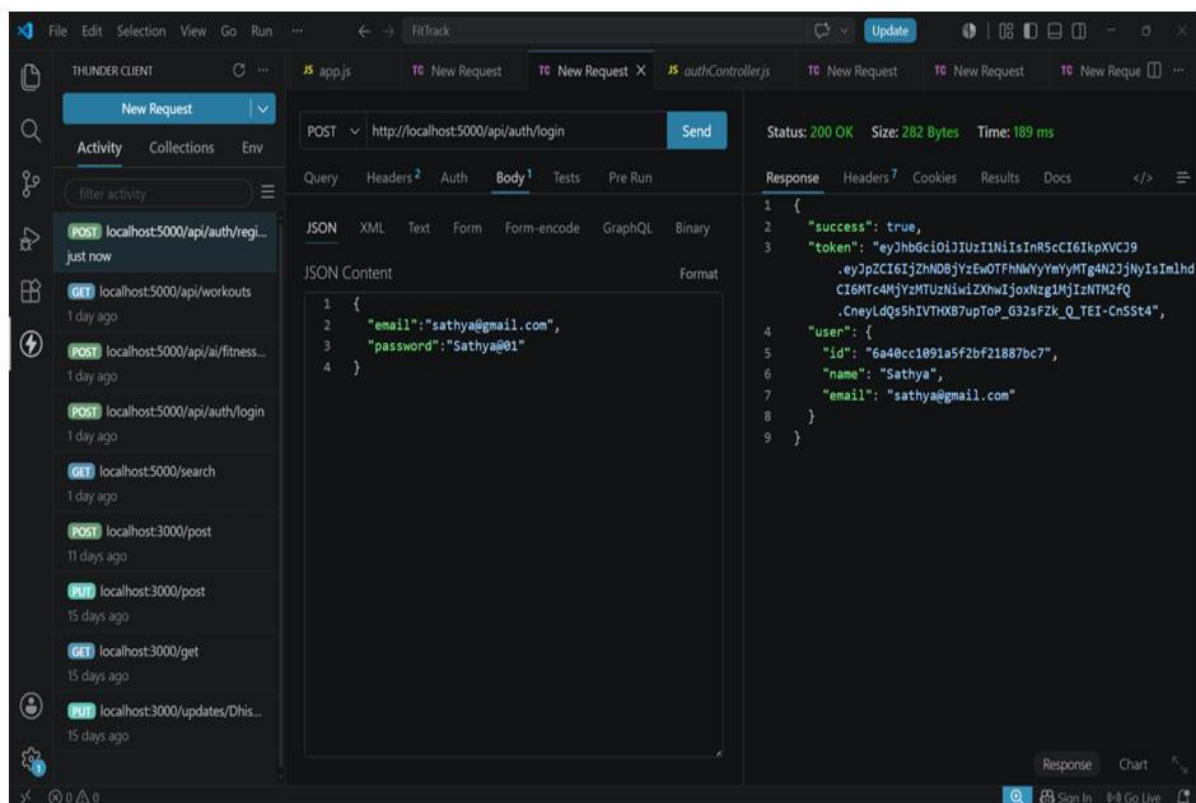
11. Screenshots

The documentation can include screenshots of the following application modules:

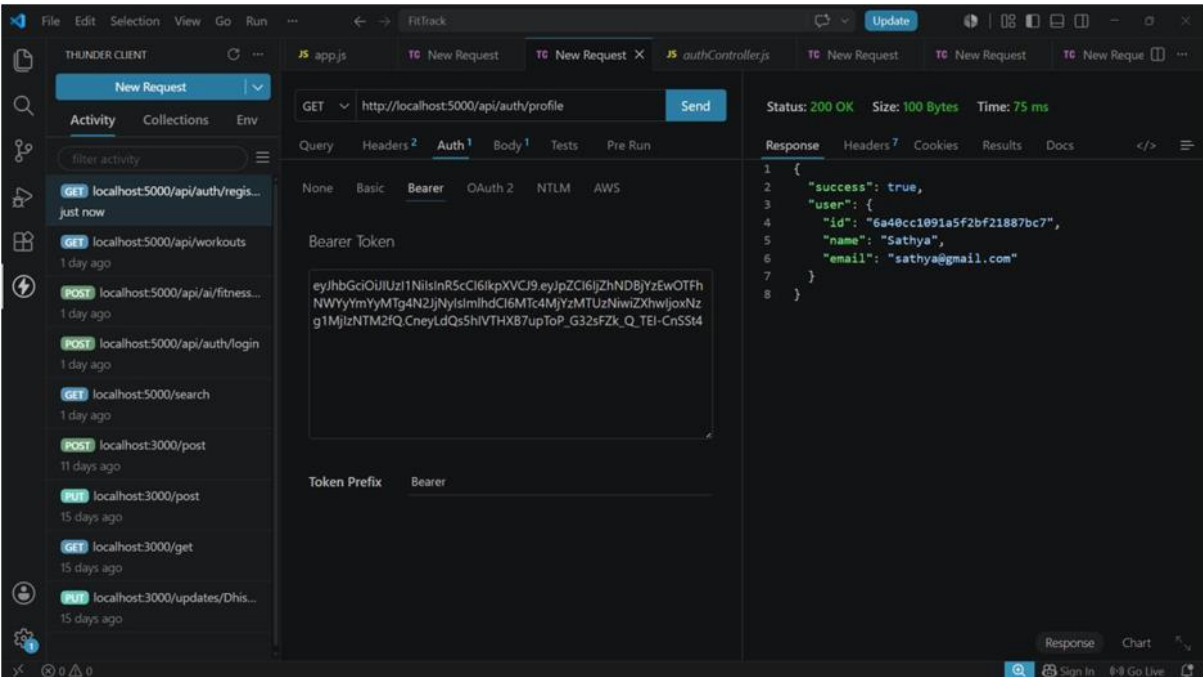
Register User



Login User

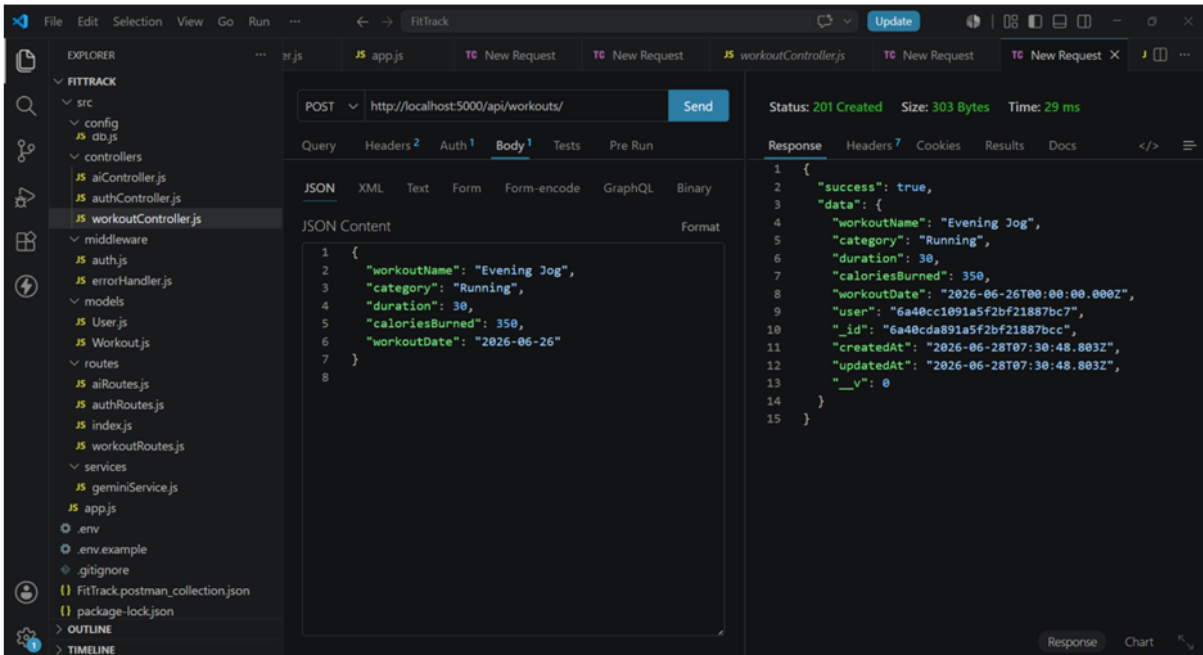


Get Profile

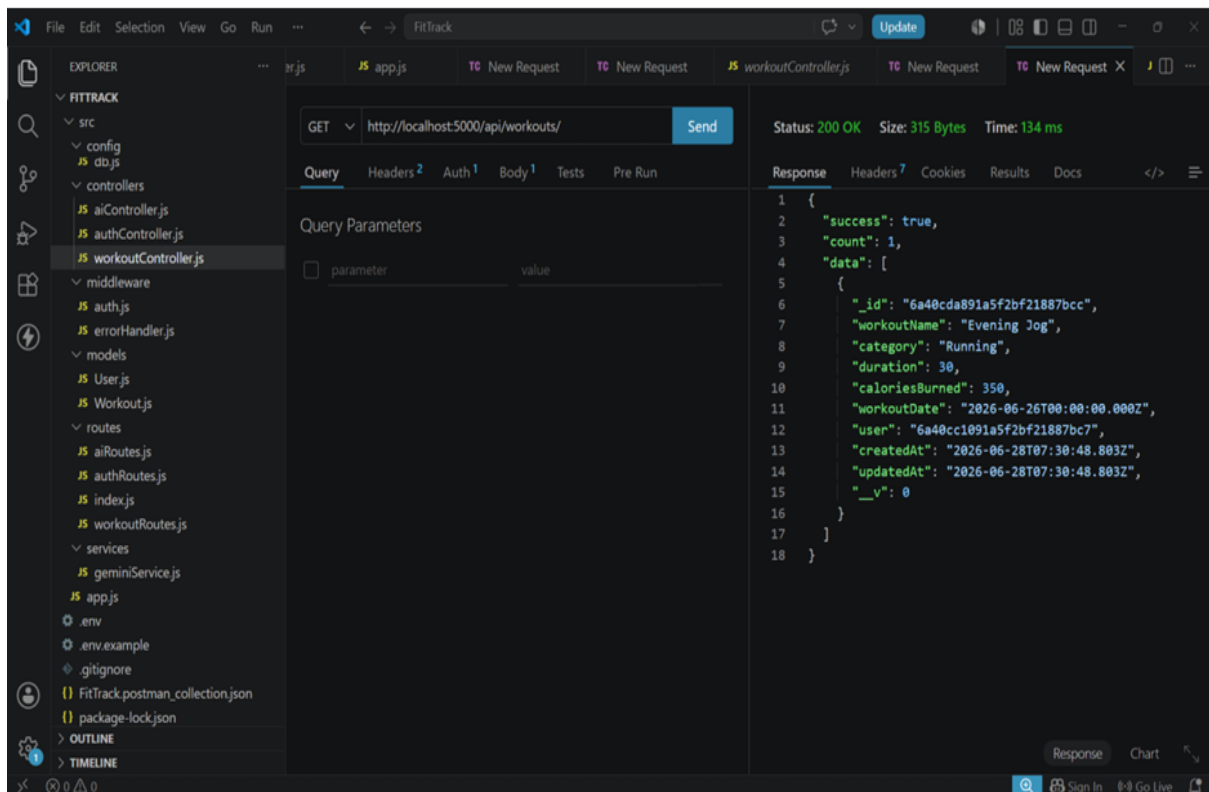


Workout API Testing

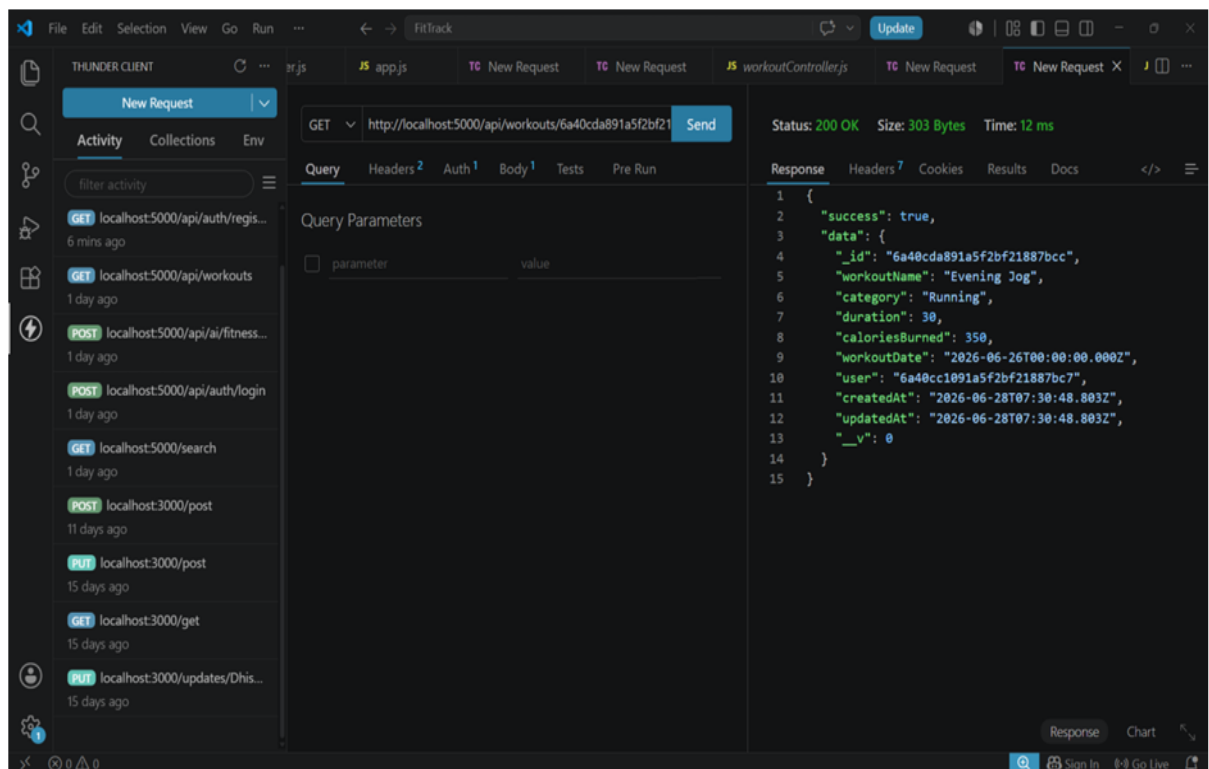
Add Workout



Get All Workouts



Get Workout By ID



Update Workout

The screenshot shows the Thunder Client interface with a PUT request to `http://localhost:5000/api/workouts/6a40cda891a5f2bf21`. The request body is a JSON object representing a workout. The response is a 200 OK status with a JSON body indicating success.

Request:

```
PUT http://localhost:5000/api/workouts/6a40cda891a5f2bf21
```

JSON Content:

```
{  "workoutName": "Evening Jog",  "category": "Running",  "duration": 60,  "caloriesBurned": 650,  "workoutDate": "2026-06-26"}
```

Response:

```
{  "success": true,  "data": {    "_id": "6a40cda891a5f2bf21887bcc",    "workoutName": "Evening Jog",    "category": "Running",    "duration": 60,    "caloriesBurned": 650,    "workoutDate": "2026-06-26T00:00:00.000Z",    "user": "6a40cc1091a5f2bf21887bc7",    "createdAt": "2026-06-28T07:30:48.803Z",    "updatedAt": "2026-06-28T07:33:42.915Z",    "__v": 0  }  }}
```

Delete Workout

The screenshot shows the Thunder Client interface with a DELETE request to `http://localhost:5000/api/workouts/6a40cda891a5f2bf21`. The response is a 200 OK status with a JSON body indicating success.

Request:

```
DELETE http://localhost:5000/api/workouts/6a40cda891a5f2bf21
```

Query Parameters:

parameter	value
-----------	-------

Response:

```
{  "success": true,  "message": "Workout removed successfully"}
```

12. Known Issues

The following limitations may be present in the current version:

- Internet connectivity is required for Google Gemini API requests.
- AI responses depend on the availability and response of the Gemini API.
- The system requires correct environment variable configuration.
- MongoDB must be available for database operations.
- AI-generated recommendations should be treated as general fitness guidance rather than professional medical advice.
- The current system may not include wearable-device integration or advanced analytics.

13. Future Enhancements

The following features can be added in future versions:

1. Nutrition Tracking

- Food and calorie tracking.
- Personalized nutrition suggestions.

2. BMI Calculator

- Calculate and monitor BMI.

3. Wearable Integration

- Connect fitness watches and other wearable devices.

4. Workout Reminders

- Notifications for scheduled workouts.

5. Analytics Dashboard

- Charts and graphs for fitness progress.

6. Advanced AI

- More personalized training plans based on workout history.

7. Additional Fitness Services

- Expand the application with more fitness-related features.

14. Conclusion

AI FitTrack provides a secure and scalable platform for fitness tracking and personalized workout assistance. The application combines **RESTful APIs, MongoDB, JWT authentication, secure password hashing, and Google Gemini AI** to provide users with workout management and intelligent fitness recommendations.

The modular architecture makes the system easier to maintain and extend. Future enhancements such as nutrition tracking, wearable integration, reminders, and advanced analytics can further expand the capabilities of AI FitTrack.